

UCM Competitive Programming Handbook

Beta - misma fuente Markdown para Web y PDF

UCM Competitive Programming

Índice

1	UCM Competitive Programming Handbook	1
1.1	Contenido de esta beta	1
1.2	Filosofía	1
1.3	Plantilla C++ mínima	2
2	Complejidad algorítmica	2
2.1	Referencia rápida	2
2.2	Orden de crecimiento	2
2.3	Regla para competencia	2
3	Fenwick Tree	2
3.1	Operaciones	2
3.2	Idea clave	3
3.3	Implementación	3
3.4	Cuándo usarlo	3
4	Dijkstra	3
4.1	Propiedades	3
4.2	Implementación	4
4.3	Errores frecuentes	4
4.4	Checklist de competencia	4

1 UCM Competitive Programming Handbook

Esta es una **beta mínima** para comprobar que un mismo conjunto de archivos Markdown puede alimentar dos salidas:

1. una web navegable con MkDocs;
2. un handbook PDF offline con Pandoc.

1.1 Contenido de esta beta

Área	Tema	Complejidad principal	Uso típico
Fundamentos	Notación Big-O	-	Estimar si una solución entra en tiempo
Estructuras	Fenwick Tree	$O(\log n)$	Sumas prefijas con actualizaciones
Grafos	Dijkstra	$O((V+E) \log V)$	Caminos mínimos con pesos no negativos

1.2 Filosofía

El directorio `content/` es la **fuentes única de verdad**. Si corriges una explicación o un código aquí, el cambio aparecerá tanto en la web como en el siguiente PDF generado.

1.3 Plantilla C++ mínima

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

2 Complejidad algorítmica

Antes de implementar una solución conviene comparar el tamaño máximo de entrada con la complejidad esperada.

2.1 Referencia rápida

n aproximado	Complejidad que suele ser razonable
$n \leq 20$	$O(2^n)$ o backtracking
$n \leq 500$	$O(n^3)$ en algunos problemas
$n \leq 5,000$	$O(n^2)$
$n \leq 200,000$	$O(n \log n)$
$n \leq 1,000,000$	$O(n)$ o cercano

Estas cifras son orientativas: el límite real depende del lenguaje, las constantes y el tiempo disponible.

2.2 Orden de crecimiento

De menor a mayor costo típico:

```
O(1)
O(log n)
O(n)
O(n log n)
O(n^2)
O(n^3)
O(2^n)
O(n!)
```

2.3 Regla para competencia

Si el problema permite $n = 2 \cdot 10^5$, una solución $O(n^2)$ casi siempre es una señal de que falta una optimización o una idea distinta.

3 Fenwick Tree

Un **Fenwick Tree** o **Binary Indexed Tree (BIT)** mantiene información acumulada y permite actualizaciones puntuales y consultas de prefijo en $O(\log n)$.

3.1 Operaciones

Operación	Complejidad
Actualizar una posición	$O(\log n)$
Suma de prefijo $[1, i]$	$O(\log n)$
Suma de rango $[l, r]$	$O(\log n)$
Memoria	$O(n)$

3.2 Idea clave

La expresión:

```
i & -i
```

obtiene el bit menos significativo encendido. Ese valor determina el salto entre nodos internos del árbol.

3.3 Implementación

```
struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int n) : n(n), bit(n + 1, 0) {}

    void add(int i, long long delta) {
        for (; i <= n; i += i & -i)
            bit[i] += delta;
    }

    long long sumPrefix(int i) const {
        long long ans = 0;
        for (; i > 0; i -= i & -i)
            ans += bit[i];
        return ans;
    }

    long long sumRange(int l, int r) const {
        return sumPrefix(r) - sumPrefix(l - 1);
    }
};
```

3.4 Cuándo usarlo

Es una buena opción cuando necesitas muchas operaciones mezcladas de:

- actualizar $a[i]$;
- consultar sumas prefijas;
- consultar sumas de intervalos.

Para operaciones más generales sobre rangos, un Segment Tree puede ser más flexible.

4 Dijkstra

El algoritmo de **Dijkstra** calcula las distancias mínimas desde un nodo origen cuando las aristas tienen pesos **no negativos**.

4.1 Propiedades

Elemento	Valor
Tipo de grafo	Dirigido o no dirigido
Pesos negativos	No permitidos
Estructura habitual	<code>priority_queue</code>
Complejidad	$O((V + E) \log V)$

4.2 Implementación

```
using ll = long long;
using Edge = pair<int, int>; // {to, weight}

vector<ll> dijkstra(int source, const vector<vector<Edge>>& g) {
    const ll INF = (1LL << 62);
    int n = (int)g.size();

    vector<ll> dist(n, INF);
    priority_queue<pair<ll, int>,
                  vector<pair<ll, int>>,
                  greater<pair<ll, int>>> pq;

    dist[source] = 0;
    pq.push({0, source});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d != dist[u])
            continue;

        for (auto [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}
```

4.3 Errores frecuentes

- usar Dijkstra cuando existen pesos negativos;
- usar `int` si la suma de pesos puede superar $2^{31} - 1$;
- olvidar descartar entradas viejas de la cola de prioridad;
- elegir un `INF` demasiado pequeño.

4.4 Checklist de competencia

1. ¿Todos los pesos son no negativos?
2. ¿Las distancias necesitan `long long`?
3. ¿El grafo es dirigido o no dirigido?
4. ¿Solo necesitas una fuente o varias?